

Android 瀑布流实现

Android 瀑布流列表实现（XML + Java）

瀑布流（Waterfall Flow）是一种不规则网格布局，特点是**列宽一致、行高自适应**，常用于图片、商品卡片等展示场景。下面分「原生实现」和「第三方库实现」两部分，用简单明了的方式讲解核心逻辑。

一、原生实现（RecyclerView + StaggeredGridLayoutManager）

Android 原生通过 `RecyclerView` + 自带的 `StaggeredGridLayoutManager` 就能实现瀑布流，无需额外依赖，核心是「布局管理器配置」+「子项高度随机/自适应」。

1. 核心原理

- `RecyclerView`：负责列表的复用和数据绑定（替代传统 `ListView`）；
- `StaggeredGridLayoutManager`：`RecyclerView` 的布局管理器，专门用于「瀑布流/网格流」，支持设置列数、方向（垂直/水平）；
- 子项布局：每个 Item 的高度不固定（根据内容或随机生成），布局管理器会自动将 Item 填充到当前最短的列。

2. 步骤实现（4步搞定）

步骤1：添加 RecyclerView 依赖（若未添加）

在 `app/build.gradle`（Module 级别）的 `dependencies` 中添加：

Code block

```
1 implementation 'androidx.recyclerview:recyclerview:1.3.2' // 最新版本可自行替换
```

同步后即可使用 `RecyclerView`。

步骤2：XML 布局文件

- 主布局（`activity_main.xml`）：放置 `RecyclerView`

Code block

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
```

```

4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical">
7
8      <!-- 瀑布流列表 -->
9      <androidx.recyclerview.widget.RecyclerView
10         android:id="@+id/recycler_view"
11         android:layout_width="match_parent"
12         android:layout_height="match_parent"
13         android:padding="8dp"/> <!-- 可选：添加内边距，避免Item贴边 -->
14
15 </LinearLayout>

```

- 子项布局 (item_waterfall.xml)：每个列表项的布局（以“图片+文字”为例）

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content" <!-- 高度自适应 -->
6      android:orientation="vertical"
7      android:padding="8dp"
8      android:background="@android:color/white"
9      android:layout_margin="4dp"> <!-- Item之间的间距 -->
10
11      <!-- 图片（瀑布流核心：图片高度不固定） -->
12      <ImageView
13          android:id="@+id/iv_img"
14          android:layout_width="match_parent"
15          android:layout_height="wrap_content" <!-- 高度由图片或代码控制 -->
16          android:scaleType="centerCrop"/>
17
18      <!-- 文字描述 -->
19      <TextView
20          android:id="@+id/tv_desc"
21          android:layout_width="match_parent"
22          android:layout_height="wrap_content"
23          android:layout_marginTop="8dp"
24          android:textSize="14sp"/>
25
26 </LinearLayout>

```

步骤3：Java 代码（数据模型 + Adapter + Activity）

(1) 数据模型 (WaterfallItem.java) : 存储每个Item的数据

Code block

```
1  public class WaterfallItem {
2      private int imgResId; // 图片资源ID
3      private String desc; // 文字描述
4
5      // 构造方法
6      public WaterfallItem(int imgResId, String desc) {
7          this.imgResId = imgResId;
8          this.desc = desc;
9      }
10
11     // getter
12     public int getImgResId() { return imgResId; }
13     public String getDesc() { return desc; }
14 }
```

(2) RecyclerView 适配器 (WaterfallAdapter.java) : 绑定数据和视图

Code block

```
1  import android.view.LayoutInflater;
2  import android.view.View;
3  import android.view.ViewGroup;
4  import android.widget.ImageView;
5  import android.widget.TextView;
6  import androidx.annotation.NonNull;
7  import androidx.recyclerview.widget.RecyclerView;
8  import java.util.List;
9
10 public class WaterfallAdapter extends
11     RecyclerView.Adapter<WaterfallAdapter.MyViewHolder> {
12
13     // 数据集合
14     private List<WaterfallItem> mData;
15
16     // 构造方法: 接收数据
17     public WaterfallAdapter(List<WaterfallItem> data) {
18         this.mData = data;
19     }
20
21     // 1. 创建ViewHolder (加载子项布局)
22     @NonNull
23     @Override
```

```

23     public MyViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
24         View itemView = LayoutInflater.from(parent.getContext())
25             .inflate(R.layout.item_waterfall, parent, false);
26         return new MyViewHolder(itemView);
27     }
28
29     // 2. 绑定数据 (给子项控件赋值)
30     @Override
31     public void onBindViewHolder(@NonNull MyViewHolder holder, int position) {
32         WaterfallItem item = mData.get(position);
33         // 设置图片 (这里用随机高度模拟瀑布流, 实际可根据图片比例计算)
34         holder.ivImg.setImageResource(item.getImgResId());
35         // 随机设置图片高度 (核心: 制造不规则行高)
36         int randomHeight = (int) (300 + Math.random() * 200); // 300~500px
37         ViewGroup.LayoutParams params = holder.ivImg.getLayoutParams();
38         params.height = randomHeight;
39         holder.ivImg.setLayoutParams(params);
40
41         // 设置文字
42         holder.tvDesc.setText(item.getDesc());
43     }
44
45     // 3. 获取数据总数
46     @Override
47     public int getItemCount() {
48         return mData.size();
49     }
50
51     // ViewHolder: 持有子项控件引用 (复用)
52     static class MyViewHolder extends RecyclerView.ViewHolder {
53         ImageView ivImg;
54         TextView tvDesc;
55
56         public MyViewHolder(@NonNull View itemView) {
57             super(itemView);
58             ivImg = itemView.findViewById(R.id.iv_img);
59             tvDesc = itemView.findViewById(R.id.tv_desc);
60         }
61     }
62 }

```

(3) Activity 中初始化 (MainActivity.java) : 设置布局管理器和适配器

Code block

```
1 import android.os.Bundle;
```

```

2  import androidx.appcompat.app.AppCompatActivity;
3  import androidx.recyclerview.widget.RecyclerView;
4  import androidx.recyclerview.widget.StaggeredGridLayoutManager;
5  import java.util.ArrayList;
6  import java.util.List;
7
8  public class MainActivity extends AppCompatActivity {
9
10     private RecyclerView mRecyclerView;
11     private WaterfallAdapter mAdapter;
12     private List<WaterfallItem> mData;
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         setContentView(R.layout.activity_main);
18
19         // 1. 初始化RecyclerView
20         mRecyclerView = findViewById(R.id.recycler_view);
21
22         // 2. 设置瀑布流布局管理器
23         int spanCount = 2; // 列数 (这里设2列, 可修改)
24         // 参数1: 列数; 参数2: 布局方向 (VERTICAL=垂直瀑布流, HORIZONTAL=水平瀑布流)
25         StaggeredGridLayoutManager layoutManager = new
StaggeredGridLayoutManager(spanCount, StaggeredGridLayoutManager.VERTICAL);
26         mRecyclerView.setLayoutManager(layoutManager);
27
28         // 3. 准备测试数据 (替换成自己的图片资源和文字)
29         mData = new ArrayList<>();
30         for (int i = 0; i < 20; i++) {
31             // 假设drawable中有img1、img2、img3三张图片, 循环使用
32             int imgRes = i % 3 == 0 ? R.drawable.img1 : (i % 3 == 1 ?
R.drawable.img2 : R.drawable.img3);
33             mData.add(new WaterfallItem(imgRes, "瀑布流 Item " + (i + 1)));
34         }
35
36         // 4. 设置适配器
37         mAdapter = new WaterfallAdapter(mData);
38         mRecyclerView.setAdapter(mAdapter);
39     }
40 }

```

3. 关键说明

- 列数控制: `StaggeredGridLayoutManager(spanCount, ...)` 中 `spanCount` 为列数 (如3列则设为3) ;

- 高度控制：示例中用 `Math.random()` 生成随机高度，实际开发中应根据图片原始比例（宽高比）计算高度，避免图片拉伸；
- 间距调整：可通过 `ItemDecoration` 给 Item 增加统一间距（原生支持，需自定义）。

二、第三方库实现（推荐2个轻量库）

原生实现足够满足基础需求，但如果需要「下拉刷新、上拉加载、Header/Footer、动画效果」等高级功能，推荐使用成熟第三方库，减少重复开发。

1. 推荐库1：AndroidStaggeredGrid

- 简介：专门用于瀑布流的库，基于 RecyclerView 封装，支持垂直/水平瀑布流、自定义间距、点击事件等；
- 依赖：

Code block

```
1 implementation 'com.etsy.android.grid:library:1.0.5'
```

- 核心用法（XML + Java）：

Code block

```
1 <!-- XML布局：直接使用StaggeredGridView -->
2 <com.etsy.android.grid.StaggeredGridView
3     android:id="@+id/staggered_grid"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     app:column_count="2" <!-- 列数 -->
7     app:item_margin="8dp"/> <!-- Item间距 -->
```

Code block

```
1 // Java代码：设置适配器（类似ListView Adapter）
2 StaggeredGridView gridView = findViewById(R.id.staggered_grid);
3 gridView.setAdapter(new WaterfallAdapter(this, mData)); // 适配器可复用原生逻辑
```

2. 推荐库2：BRVAH（BaseRecyclerViewAdapterHelper）

- 简介：最流行的 RecyclerView 适配器辅助库，支持瀑布流、网格流、线性流，自带下拉刷新、上拉加载、空布局、多类型Item等；
- 依赖：

```
implementation 'com.github.CymChad:BaseRecyclerViewAdapterHelper:3.0.4'
```

- 核心用法（简化适配器代码）：

Code block

```
1  // 1. 定义适配器 (无需写ViewHolder, 库已封装)
2  public class MyWaterfallAdapter extends BaseQuickAdapter<WaterfallItem,
    BaseViewHolder> {
3      public MyWaterfallAdapter(List<WaterfallItem> data) {
4          super(R.layout.item_waterfall, data);
5      }
6
7      // 绑定数据 (自动复用ViewHolder)
8      @Override
9      protected void convert(BaseViewHolder helper, WaterfallItem item) {
10         ImageView ivImg = helper.getView(R.id.iv_img);
11         // 计算图片高度 (示例用随机高度)
12         int randomHeight = (int) (300 + Math.random() * 200);
13         ViewGroup.LayoutParams params = ivImg.getLayoutParams();
14         params.height = randomHeight;
15         ivImg.setLayoutParams(params);
16
17         helper.setImageResource(R.id.iv_img, item.getImgResId());
18         .setText(R.id.tv_desc, item.getDesc());
19     }
20 }
21
22 // 2. Activity中使用
23 RecyclerView recyclerView = findViewById(R.id.recycler_view);
24 recyclerView.setLayoutManager(new StaggeredGridLayoutManager(2,
    StaggeredGridLayoutManager.VERTICAL));
25 MyWaterfallAdapter adapter = new MyWaterfallAdapter(mData);
26 recyclerView.setAdapter(adapter);
27
28 // 自带上拉加载 (一行代码)
29 adapter.getLoadMoreModule().setOnLoadMoreListener(() -> {
30     // 加载更多数据逻辑
31     loadMoreData();
32 });
```

三、总结

实现方式	优点	缺点	适用场景
------	----	----	------

原生（RecyclerView）	无依赖、轻量、灵活控制	需自己实现下拉刷新、上拉加载	基础瀑布流需求
第三方库（BRVAH等）	自带高级功能、简化代码、稳定	引入额外依赖	复杂需求（加载更

新手建议先掌握原生实现，理解瀑布流核心逻辑；实际项目中如果需要高级功能，直接使用 BRVAH（功能最全）或 AndroidStaggeredGrid（专注瀑布流）即可。

（注：文档部分内容可能由 AI 生成）